

# SMART TIMETABLE & ROOM ALLOCATION MANAGEMENT SYSTEM

Project Requirement, System Design & Development Specification

|                         |                                |
|-------------------------|--------------------------------|
| <b>Backend</b>          | Django + Django REST Framework |
| <b>Frontend</b>         | Next.js + TypeScript           |
| <b>Database</b>         | PostgreSQL                     |
| <b>Background Jobs</b>  | Celery + Redis                 |
| <b>Document Version</b> | 1.0                            |
| <b>Date</b>             | 21 September 2026              |

*Prepared from the user-supplied BBDU timetable references and extended into a complete web application specification.*

## 1. Executive Summary

This project is a web-based academic timetable and room allocation system for colleges/universities. It will let authorised academic staff maintain master data, generate or manually edit class timetables, allocate classrooms/labs, detect conflicts, publish approved versions, notify faculty members when a timetable changes, and generate downloadable PDFs that can be shared with student groups.

The system should treat a published timetable as a versioned official document. When an already-published timetable is changed, the application creates a new revision, shows the differences, validates faculty/section/room conflicts, requires approval if configured, regenerates class and room-allocation PDFs, and sends update information to faculty. A permanent “latest timetable” share link can always point to the newest published version.

**Core outcome:** One source of truth for section schedules, faculty schedules and room utilisation, with automatic conflict checking and reliable communication after every approved update.

## 2. What the Reference Timetables Show

The uploaded reference PDFs are image-based timetable sheets. Their visual structure is consistent across multiple sections and years. The proposed application should preserve the useful characteristics of this format while making the data structured and editable.

| Observed element      | System interpretation                                                                                       |
|-----------------------|-------------------------------------------------------------------------------------------------------------|
| Header                | University/School/Department, programme year, semester and academic session.                                |
| Weekly matrix         | Monday-Friday rows with hourly periods such as 09-10, 10-11, 11-12, 12-1, lunch, 2-3, 3-4 and 4-5.          |
| Compact slot notation | Cells combine activity type/course or subject shorthand/faculty initials/room or lab number.                |
| Class details         | Each sheet identifies section and class coordinator, and may show coordinator contact details.              |
| Course mapping        | A lower table maps credits, course codes, course names, metadata/short notation and faculty name.           |
| Approval footer       | Official sheets include academic authority names/signatures.                                                |
| Scale                 | The files contain many section-specific sheets, so bulk generation and consistent formatting are important. |

**Important implementation choice:** Do not store timetable cells as raw strings such as “L/CAIT/SY/411”. Store course, faculty, room, class type and time slot as separate database relations, then generate the compact display notation at render time.

## 3. Project Goals

- Create section-wise timetables for all programmes, years and sections.
- Automatically assign eligible rooms/labs and produce a room allotment chart.
- Prevent faculty, section and room double-booking before publication.
- Support both automatic timetable generation and manual drag-and-drop editing.
- Maintain faculty availability and workload limits.
- Publish versioned timetables with approval and audit history.
- Notify all faculty when a new official version is published; send detailed change information to affected faculty.
- Provide one-click PDF download and shareable latest-version links for class coordinators/faculty.
- Generate faculty-wise, section-wise and room-wise views from the same schedule data.
- Make master-data setup practical through CSV/Excel import and export.

## 4. User Roles & Permissions

| Role        | Main permissions                                                                       |
|-------------|----------------------------------------------------------------------------------------|
| Super Admin | System configuration, institutions/campuses, global users, all reports and audit logs. |

| Role                                   | Main permissions                                                                                  |
|----------------------------------------|---------------------------------------------------------------------------------------------------|
| Academic Admin / Timetable Coordinator | Master data, generation, editing, validation, version creation, publication and notifications.    |
| HOD / Dean / Approver                  | Review draft/revision, view conflict report, approve or reject publication.                       |
| Class Coordinator                      | View/download section timetable, receive updates, share official link/PDF, report issues.         |
| Faculty                                | View personal timetable and changed slots, acknowledge updates, download class/faculty timetable. |
| Room/Lab Coordinator                   | Maintain room/lab availability, maintenance blocks and room utilisation.                          |
| Read-only Viewer                       | Optional internal user who can view published timetables but cannot edit.                         |

## 5. Functional Modules

| Module                        | Scope                                                                                                                                 |
|-------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| Authentication & RBAC         | Login, JWT/session handling, role permissions, password reset, profile and institution/department scope.                              |
| Academic Master Data          | Academic session, odd/even semester, programme, year, section, department, batch and student strength.                                |
| Course & Offering Setup       | Course code/name/credit, lecture/tutorial/practical requirements, weekly periods, assigned faculty, preferred rooms/labs.             |
| Faculty Management            | Faculty profile, initials, department, availability, unavailable slots, workload limits and optional preferred slots.                 |
| Room & Lab Management         | Room code, building/floor, capacity, type, facilities, lab type, availability and maintenance blocks.                                 |
| Time Slot Template            | Working days, period start/end, lunch/break slots, optional Saturday, locked common periods and holiday overrides.                    |
| Automatic Timetable Generator | Constraint-based generation for selected departments/years/sections, preserving locked entries and reporting unsatisfied constraints. |
| Visual Timetable Builder      | Grid editor with click/drag assignment, filters, conflict indicators, copy day/week, lock slot and undo/redo.                         |
| Room Allocation Chart         | Daily/weekly room matrix, occupied/free rooms, class/subject/faculty per room and room conflict prevention.                           |
| Validation & Conflict Engine  | Faculty clash, room clash, section clash, room suitability/capacity, faculty availability, weekly period count and lab continuity.    |
| Versioning & Approval         | Draft, validation, approval, publish, supersede, diff and rollback/clone to new revision.                                             |
| Notifications                 | In-app, email, optional WhatsApp/SMS/push; publish summary to all faculty and detailed diffs to affected faculty/coordinators.        |
| PDF / Print / Share           | Official section PDF, faculty PDF, room allotment PDF, combined batch PDF, print layout, QR/share link and latest-version URL.        |
| Reports & Analytics           | Faculty workload, room utilisation, free rooms, course-period completion, unresolved conflicts and publication history.               |
| Audit & Activity Log          | Who changed what, old/new values, publish time, approval history, generated files and delivery status.                                |

## 6. Core Scheduling Rules

### 6.1 Hard constraints (must never be violated)

- A section cannot attend two classes in the same time slot.
- A faculty member cannot teach two schedule entries in the same time slot, even across different sections.
- A room/lab cannot be assigned to two classes in the same time slot.
- A scheduled entry must use a room that is active and available for that slot.
- Room capacity should be greater than or equal to the section strength unless an authorised override is recorded.
- A practical/lab course must use a compatible lab/room type and required facilities.
- Faculty unavailable slots cannot be used.
- Locked/common periods (lunch, university activity, GP, NSS/Yoga, etc.) must remain fixed when configured.
- Required consecutive blocks for labs/practicals must stay contiguous.
- Published versions cannot be edited in place; changes are made in a new revision.

### 6.2 Soft constraints (optimisation preferences)

- Minimise unnecessary gaps between faculty lectures.
- Avoid too many consecutive lectures for the same faculty or section.
- Spread a course across the week instead of placing all periods on one day.
- Prefer the same classroom for a section when practical.
- Prefer faculty-selected time windows where possible.
- Balance room utilisation and avoid assigning oversized rooms to very small sections.
- Avoid first/last period for selected courses if configured.
- Keep practical blocks in preferred parts of the day.

## 7. Automatic Timetable Generation Strategy

Use Google OR-Tools CP-SAT in the Django backend for the automatic generator. Django should prepare the scheduling problem from database master data, send long-running generation to a Celery worker, and persist the best valid solution as a draft timetable version.

1. Load selected academic session, semester, sections, course offerings, faculty, rooms and time slots.
2. Convert every required weekly course period/block into scheduling variables.
3. Add hard constraints for section, faculty, room, availability, room type/capacity and locked slots.
4. Add weighted soft constraints for workload balance, gaps, distribution and preferred rooms/times.
5. Run CP-SAT with a configurable time limit and progress state.
6. Save a draft result plus a generation report containing score, warnings and any unmet optional preferences.
7. Allow the coordinator to manually adjust entries while retaining live conflict validation.
8. Publish only after zero blocking conflicts and required approval.

**Practical MVP approach:** Build manual scheduling + conflict validation first, then add CP-SAT generation on top of the same data model. This reduces risk and lets the system deliver value before automatic optimisation is perfect.

## 8. Timetable Version & Update Workflow

| Stage            | Behaviour                                                                                                                                   |
|------------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| 1. Create        | Coordinator creates a new timetable or clones the latest published version into a draft.                                                    |
| 2. Edit/Generate | Automatic generator or visual builder fills schedule entries. Locked entries remain fixed.                                                  |
| 3. Validate      | Backend returns blocking conflicts and warnings. Publish button stays disabled while blockers exist.                                        |
| 4. Review        | Optional HOD/Dean approval with comments.                                                                                                   |
| 5. Publish       | Version number is assigned, official PDFs are generated and the version becomes immutable.                                                  |
| 6. Notify        | All faculty receive a new-version notice. Affected faculty and class coordinators receive detailed changed slots plus download/share links. |

| Stage     | Behaviour                                                                                                           |
|-----------|---------------------------------------------------------------------------------------------------------------------|
| 7. Share  | Faculty downloads the PDF or shares the permanent latest-version link with student groups.                          |
| 8. Revise | Any later change creates a new draft revision; the system shows old vs new and repeats validation/approval/publish. |

## 9. Change Detection & Notification Behaviour

Every publication should calculate a structured diff between the previous published version and the new version. A change item should identify day, slot, section, course, old/new faculty, old/new room and whether the entry was added, removed or moved.

| Recipient                       | Notification content                                                                                      |
|---------------------------------|-----------------------------------------------------------------------------------------------------------|
| All faculty                     | "Timetable version Vx published" summary, effective date, download link, latest-version link.             |
| Affected faculty                | Detailed changed periods for that faculty, old → new values, personal timetable link, acknowledge button. |
| Class coordinator               | Detailed changes for their section and section PDF/share link.                                            |
| HOD/Dean                        | Publication/approval summary and unresolved non-blocking warnings, if any.                                |
| Optional student/public channel | No direct personal data; expose only the published section timetable through a controlled share link/QR.  |

### 9.1 Notification channels

- In-app notification centre with unread/read state.
- Email containing a concise change summary and secure download/share links.
- Optional WhatsApp Cloud API integration for faculty broadcast/individual updates.
- Optional PWA/web push notifications for instant browser alerts.
- Delivery log with sent/failed/retry status; Celery retries transient failures.

## 10. Room Allocation Module

Room allocation must be generated from the same ScheduleEntry records as the class timetable. This prevents separate room charts from drifting out of sync with the actual timetable.

- Weekly room chart: rows = rooms, columns = periods, filtered by day; or rows = periods, columns = rooms.
- Cell content: section, course/short code, faculty initials and room status.
- Statuses: Occupied, Free, Maintenance/Blocked and Conflict (draft validation only).
- Filters: department, building, floor, room type, lab type, capacity and date/day.
- "Find a free room" action for a selected day/slot and expected student count.
- Room utilisation percentage by week and time band.
- Dedicated downloadable Room Allotment Chart PDF with version, effective date and generated timestamp.
- When a timetable slot changes, the room chart updates automatically because it is not a separate source of data.

## 11. Proposed Data Model

| Entity              | Important fields                                                   | Purpose                                              |
|---------------------|--------------------------------------------------------------------|------------------------------------------------------|
| Institution         | name, code, logo, timezone                                         | Top-level scope; future multi-campus support.        |
| Department          | institution_id, name, code                                         | Owns programmes, faculty and rooms where applicable. |
| Program             | department_id, name, code, duration_years                          | B.Tech CSE, CSE-AI, etc.                             |
| AcademicSession     | institution_id, name, start_date, end_date, active                 | Example: 2026-27.                                    |
| Semester            | session_id, number/name, type, start_date, end_date                | Odd/Even semester context.                           |
| Section             | program_id, year, name, student_strength, coordinator_id           | Class group such as CSE-2A.                          |
| Faculty             | user_id, employee_code, initials, department_id, max_daily_periods | Teacher master.                                      |
| FacultyAvailability | faculty_id, day, slot_id, is_available, preference_weight          | Availability and preference.                         |

| Entity               | Important fields                                                                         | Purpose                               |
|----------------------|------------------------------------------------------------------------------------------|---------------------------------------|
| Course               | code, name, credit, short_code                                                           | Academic subject master.              |
| CourseOffering       | semester_id, section_id, course_id, weekly_periods, class_type, assigned faculty         | The subject as taught to one section. |
| Room                 | code, building, floor, capacity, room_type, facilities, active                           | Classroom/lab inventory.              |
| RoomAvailability     | room_id, day/date, slot_id, status, reason                                               | Maintenance or reserved blocks.       |
| TimeSlot             | template_id, day applicability, start_time, end_time, slot_order, is_break               | Reusable timetable periods.           |
| Timetable            | session/semester/department scope, title, effective date                                 | Logical timetable collection.         |
| TimetableVersion     | timetable_id, version_no, status, created_by, published_at, previous_version_id          | Immutable official revision.          |
| ScheduleEntry        | version_id, section_id, course_offering_id, day, slot/block, room_id, entry_type, locked | Atomic schedule item.                 |
| ScheduleEntryFaculty | schedule_entry_id, faculty_id, role                                                      | Supports co-teaching/practicals.      |
| Approval             | version_id, approver_id, status, comment, acted_at                                       | Review workflow.                      |
| ChangeLog            | from_version, to_version, object_type, old_json, new_json                                | Structured diff/audit.                |
| Notification         | recipient, channel, template, payload, read_at, delivery_status                          | Publication/update communication.     |
| GeneratedArtifact    | version_id, artifact_type, file, checksum, generated_at                                  | Section/faculty/room PDF files.       |
| ShareLink            | version/latest pointer, scope, token, expiry, active                                     | Controlled public/student sharing.    |

## 12. Database & Validation Constraints

- Unique section occupancy per (version, day, slot/block, section).
- Unique room occupancy per (version, day, slot/block, room).
- Faculty conflict validation across ScheduleEntryFaculty assignments for the same version/day/slot.
- CourseOffering weekly scheduled count must match the configured requirement before final publication unless an explicit exception is approved.
- Room type/facilities and capacity checks run whenever the room or course offering changes.
- Deletion of referenced master data should be protected or soft-deleted; historical timetable versions must remain readable.
- Published versions are read-only at service/API level, not only in the UI.
- All update endpoints should use database transactions and optimistic locking/version timestamps where appropriate.

## 13. Backend Architecture (Django)

| Component                     | Responsibility                                                                      |
|-------------------------------|-------------------------------------------------------------------------------------|
| Django + DRF                  | REST API, business rules, RBAC, master data, timetable services.                    |
| PostgreSQL                    | Transactional master/schedule/version/audit data.                                   |
| Redis                         | Celery broker/cache, generation job state, optional rate limits.                    |
| Celery                        | Automatic generation, PDF creation, bulk notifications and scheduled reminder jobs. |
| Django Channels (optional)    | Live publish/update events and builder collaboration/status.                        |
| OR-Tools                      | Constraint solver for automatic timetable generation.                               |
| WeasyPrint                    | Server-side official PDFs from HTML/CSS templates.                                  |
| Object storage / media        | Generated PDFs, institution logo/signatures and optional attachments.               |
| Email provider / SMTP         | Faculty email notifications.                                                        |
| WhatsApp Cloud API (optional) | Faculty update messages when enabled by institution.                                |

### 13.1 Suggested Django apps

```
apps/
  accounts/
  institutions/
  academics/
  faculty/
  rooms/
  scheduling/
  approvals/
  notifications/
  reports/
  audit/
  common/
```

### 13.2 Service layer

- `scheduling.services.validation` — conflict and completeness checks.
- `scheduling.services.generator` — OR-Tools model builder and solution persistence.
- `scheduling.services.versioning` — clone, diff, publish and supersede logic.
- `scheduling.services.rendering` — official timetable and room chart context builders.
- `notifications.services.dispatch` — channel routing and recipient calculation.
- `reports.services.workload` — faculty workload and room utilisation aggregations.

## 14. Frontend Architecture (Next.js)

Use Next.js App Router with TypeScript, server/client components as appropriate, React Query/TanStack Query for API state, React Hook Form + Zod for forms, Tailwind CSS and shadcn/ui for the interface.

| Route                    | Purpose                                                                            |
|--------------------------|------------------------------------------------------------------------------------|
| /login                   | Authentication.                                                                    |
| /dashboard               | Summary cards: active version, conflicts, pending approvals, changes, utilisation. |
| /master/academic         | Sessions, semesters, programmes, years, sections.                                  |
| /master/courses          | Courses and course offerings.                                                      |
| /master/faculty          | Faculty, availability and workload.                                                |
| /master/rooms            | Rooms/labs, capacity and availability.                                             |
| /master/time-slots       | Period template, lunch/breaks and working days.                                    |
| /timetables              | Timetable list, status, version and effective date.                                |
| /timetables/[id]/builder | Interactive section timetable builder.                                             |

| Route                     | Purpose                                                            |
|---------------------------|--------------------------------------------------------------------|
| /timetables/{id}/generate | Generator configuration and job progress.                          |
| /timetables/{id}/validate | Blocking conflicts, warnings and completeness.                     |
| /timetables/{id}/versions | Version history, diff, clone and rollback view.                    |
| /room-allocation          | Room chart, free-room finder and utilisation.                      |
| /faculty/me/timetable     | Personal weekly timetable and change history.                      |
| /notifications            | Updates, acknowledgement and downloads.                            |
| /reports                  | Faculty load, room usage, conflicts, publications.                 |
| /share/{token}            | Read-only latest/selected published timetable for student sharing. |

## 15. Timetable Builder UX

- Selector bar: session → semester → department → programme → year → section → version.
- Weekly grid with day rows and period columns matching the existing paper format.
- Course cards show class type, short course code, faculty initials and room.
- Click a cell to assign course, faculty, room and class type; drag entries when convenient.
- Red conflict badge for blocking issues; amber badge for warnings; green validation state when clean.
- Lock individual entries before auto-regeneration.
- Right-side inspector shows course requirement, periods remaining, faculty availability and suggested free rooms.
- Top actions: Save Draft, Auto Generate, Validate, Compare, Submit for Approval, Publish, Download PDF.
- Published view is read-only and clearly shows version/effective date.

## 16. Official Export Formats

| Output                                                                                                                                                    | Content                                                                                                                                                                              |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Section timetable PDF                                                                                                                                     | Mirrors the reference style: institutional header, section/year/session, weekly matrix, class coordinator, course mapping, approver/signature area, version and generated timestamp. |
| Faculty timetable PDF                                                                                                                                     | One faculty member's week across all sections, including room and course.                                                                                                            |
| Room allotment PDF                                                                                                                                        | Room-by-period chart for each day/week, plus free/blocked rooms if desired.                                                                                                          |
| Combined department PDF                                                                                                                                   | All selected sections in a single downloadable file.                                                                                                                                 |
| CSV/XLSX exports                                                                                                                                          | Master data, room utilisation, faculty workload and schedule entries for administration.                                                                                             |
| <b>PDF rule:</b> Every exported official PDF should print the timetable version and generated timestamp so faculty/students can identify outdated copies. |                                                                                                                                                                                      |

## 17. Suggested API Design

| Method   | Endpoint                                            | Purpose                             |
|----------|-----------------------------------------------------|-------------------------------------|
| GET/POST | /api/v1/academic/sessions/                          | Academic sessions                   |
| GET/POST | /api/v1/academic/sections/                          | Sections/batches                    |
| GET/POST | /api/v1/courses/                                    | Course master                       |
| GET/POST | /api/v1/course-offerings/                           | Section-course-faculty requirements |
| GET/POST | /api/v1/faculty/                                    | Faculty master                      |
| GET/POST | /api/v1/faculty/{id}/availability/                  | Faculty availability                |
| GET/POST | /api/v1/rooms/                                      | Rooms and labs                      |
| GET      | /api/v1/rooms/available/?day=&slot=&capacity=&type= | Free room finder                    |
| GET/POST | /api/v1/timetables/                                 | Timetable containers                |

| Method    | Endpoint                                               | Purpose                            |
|-----------|--------------------------------------------------------|------------------------------------|
| POST      | /api/v1/timetables/{id}/versions/                      | Create/clone draft version         |
| GET/PATCH | /api/v1/versions/{id}/entries/                         | Read/edit schedule entries         |
| POST      | /api/v1/versions/{id}/validate/                        | Conflict/completeness validation   |
| POST      | /api/v1/versions/{id}/generate/                        | Queue automatic generation         |
| GET       | /api/v1/generation-jobs/{job_id}/                      | Generation progress/result         |
| POST      | /api/v1/versions/{id}/submit/                          | Submit for approval                |
| POST      | /api/v1/versions/{id}/approve/                         | Approve/reject                     |
| POST      | /api/v1/versions/{id}/publish/                         | Publish, generate files and notify |
| GET       | /api/v1/versions/{id}/diff/?against={version_id}       | Structured change diff             |
| GET       | /api/v1/versions/{id}/exports/section/{section_id}.pdf | Section PDF                        |
| GET       | /api/v1/versions/{id}/exports/rooms.pdf                | Room allocation PDF                |
| GET       | /api/v1/me/timetable/                                  | Current faculty timetable          |
| GET       | /api/v1/notifications/                                 | Notification inbox                 |
| POST      | /api/v1/notifications/{id}/acknowledge/                | Faculty acknowledgement            |

## 18. Publish Transaction (Recommended Behaviour)

1. Acquire publish lock for the timetable.
2. Run a final blocking validation against the latest master data.
3. Confirm required approval status.
4. Calculate version diff against previous published version.
5. Mark current version as PUBLISHED and previous one as SUPERSEDED inside a transaction.
6. Queue PDF generation and notification jobs after transaction commit.
7. Update latest-version share pointer only after publication succeeds.
8. Record audit event with publisher, timestamp, version, diff summary and effective date.

## 19. Security, Reliability & Audit

- JWT authentication with refresh token rotation or secure HttpOnly cookie strategy.
- Role/department/institution scoped permissions enforced in DRF, not only hidden in frontend.
- CSRF protection if cookies are used; CORS restricted to approved frontend domains.
- Rate-limit authentication and public share endpoints.
- Signed, random, revocable share tokens; optional expiration for public links.
- Audit create/update/delete/publish/approval actions with actor and timestamp.
- Nightly database backup plus generated-artifact storage backup policy.
- Sentry/structured logging for server errors and failed notification jobs.
- Do not put faculty phone numbers in public/student exports unless explicitly allowed.

## 20. Performance & Scalability

- Use PostgreSQL indexes on version/day/slot, section, faculty and room scheduling lookups.
- Prefetch schedule relationships to avoid N+1 queries in weekly grids and exports.
- Cache latest published timetable payloads and room availability where safe.
- Run generation, PDFs and mass notifications asynchronously with Celery.
- Generate only affected PDFs after a revision when possible, while keeping combined exports reproducible.
- Store generated files with version-based immutable paths and checksums.

## 21. Development Phases

| Phase                             | Deliverables                                                                                |
|-----------------------------------|---------------------------------------------------------------------------------------------|
| Phase 1 - Foundation              | Auth/RBAC, academic masters, faculty, courses, rooms, slots, import/export.                 |
| Phase 2 - Manual Scheduling       | Timetable models, grid builder, room assignment, conflict validation, draft saving.         |
| Phase 3 - Versioning & Publishing | Approval, publish, diff, audit, PDF exports, latest share link.                             |
| Phase 4 - Notifications           | In-app + email, acknowledgement, optional WhatsApp/PWA push.                                |
| Phase 5 - Automatic Generation    | OR-Tools, locked entries, hard/soft constraints, background progress.                       |
| Phase 6 - Reports & Polish        | Workload/utilisation, free-room finder, dashboards, accessibility, mobile/PWA optimisation. |

## 22. Acceptance Criteria

| Criterion                   | Pass condition                                                                                             |
|-----------------------------|------------------------------------------------------------------------------------------------------------|
| No clashes                  | System blocks publication if any section, faculty or room has an overlapping schedule entry.               |
| Room correctness            | Room must meet configured capacity/type/facility constraints or show an authorised exception.              |
| Weekly completion           | Required periods for every active course offering are visible and validated.                               |
| Version safety              | Published timetable is immutable; a change creates a new revision with a diff.                             |
| Notification                | On publish, all faculty receive the new-version notice; affected faculty receive their change details.     |
| Download/share              | Faculty can download current PDFs and copy a latest published link for student groups.                     |
| Room chart consistency      | Room allocation PDF/view is derived from the same schedule and immediately reflects the published version. |
| Auditability                | Admin can determine who changed/published/approved a timetable and when.                                   |
| Reference-compatible output | Section PDF retains the familiar weekly matrix plus course/faculty mapping and official approval area.     |
| Responsive UI               | Core faculty and coordinator views remain usable on mobile as well as desktop.                             |

## 23. Important Edge Cases

- One class taught by multiple faculty members.
- A practical spanning two or three consecutive periods.
- Combined sections attending one common lecture.
- A room becomes unavailable after a timetable is published.
- Faculty leave requires partial reallocation without rebuilding the complete week.
- Different departments use the same shared lab/room.
- Special university common periods override normal scheduling.
- A new section/course/faculty is added mid-semester.
- Coordinator wants a temporary effective-from date for a revised timetable.
- Old PDF remains in a student group; visible version/effective date helps identify it as superseded.

## 24. Development Prompt for Codex / Coding Agent

The complete copy-ready implementation prompt is also provided as a separate Markdown file. Use the following as the project brief for an AI coding agent:

You are a senior full-stack architect and engineer. Build a production-ready academic Timetable & Room Allocation Management System using Django + Django REST Framework for the backend and Next.js App Router + TypeScript for the frontend.

### PROJECT GOAL

Create a web application for a university/college that can manage academic master data, generate section-wise weekly timetables, allocate classrooms/labs, prevent faculty/room/section clashes, maintain timetable versions, publish official revisions, notify faculty whenever an official timetable changes, and generate downloadable PDFs that faculty can share with student groups.

### REFERENCE FORMAT TO SUPPORT

The existing official timetable format is a weekly grid with Monday-Friday rows and hourly columns (for example 09-10, 10-11, 11-12, 12-1, lunch/break, 2-3, 3-4, 4-5). A cell may display compact metadata such as class type/course short code/faculty initials/room number. Below the grid, the official sheet contains course credit, course code, course name, metadata, faculty name, class coordinator, academic session, section/year and approval/signature area. The generated PDF must be able to reproduce this type of official layout without storing the cell as one raw string.

#### TECH STACK

Backend: Python, Django, Django REST Framework, PostgreSQL, SimpleJWT or secure cookie-based JWT, Celery, Redis, django-filter. Use OR-Tools CP-SAT for automatic generation. Use WeasyPrint for official HTML-to-PDF generation. Django Channels is optional for live update events.

Frontend: Next.js App Router, TypeScript, Tailwind CSS, shadcn/ui, TanStack Query, React Hook Form, Zod. Build responsive desktop/mobile UI.

#### ARCHITECTURE RULES

1. Use a modular Django app structure: accounts, institutions, academics, faculty, rooms, scheduling, approvals, notifications, reports, audit, common.
2. Keep business logic in service modules, not fat views/serializers.
3. Published timetable versions are immutable. Every modification after publication creates a new draft revision.
4. The section timetable, faculty timetable and room allocation chart must all be projections of the same ScheduleEntry data. Never maintain a separate room-allotment source of truth.
5. Use database transactions for version publication and concurrency-sensitive updates.
6. Add audit records for master-data changes, schedule changes, approval and publication.
7. Use soft deletion/protection for master data referenced by historical versions.

#### ROLES

SUPER\_ADMIN

ACADEMIC\_ADMIN / TIMETABLE\_COORDINATOR

HOD\_OR\_DEAN\_APPROVER

CLASS\_COORDINATOR

FACULTY

ROOM\_LAB\_COORDINATOR

READ\_ONLY\_VIEWER

Enforce permissions in DRF and mirror them in the UI.

#### CORE MODELS

Institution(name, code, logo, timezone)

Department(institution, name, code)

Program(department, name, code, duration\_years)

AcademicSession(institution, name, start\_date, end\_date, is\_active)

Semester(session, name/number, type ODD|EVEN, start\_date, end\_date)

Section(program, semester or current academic context, year, name, student\_strength, coordinator)

Faculty(user, employee\_code, initials, department, max\_daily\_periods, max\_weekly\_periods)

FacultyAvailability(faculty, weekday, time\_slot, is\_available, preference\_weight)

Course(code, name, credit, short\_code)

CourseOffering(semester, section, course, weekly\_periods, default\_class\_type, required\_block\_size, room\_type\_requirement, preferred\_room, active)

CourseOfferingFaculty(course\_offering, faculty, role, priority)

Room(code, building, floor, capacity, room\_type, facilities JSON/reasons, active)

RoomAvailability(room, weekday/date, time\_slot, status AVAILABLE|BLOCKED|MAINTENANCE, reason)

TimeSlotTemplate(name, institution)

TimeSlot(template, label, start\_time, end\_time, order, is\_break)

Timetable(session, semester, department/scope, title, effective\_date)

TimetableVersion(timetable, version\_no, status DRAFT|PENDING\_APPROVAL|APPROVED|PUBLISHED|SUPERSEDED, previous\_version, created\_by, published\_by, published\_at, notes)

ScheduleEntry(version, section, course\_offering, weekday, start\_slot, block\_length, room, entry\_type LECTURE|TUTORIAL|PRACTICAL|COMMON|LIBRARY|OTHER, locked, note)

ScheduleEntryFaculty(schedule\_entry, faculty, role)

Approval(version, approver, status, comment, acted\_at)

ChangeLog(from\_version, to\_version, object\_type, object\_id, change\_type, old\_data JSON, new\_data JSON)

Notification(recipient, channel, title, body, payload JSON, delivery\_status, read\_at, acknowledged\_at)

GeneratedArtifact(version, artifact\_type, scope\_id, file, checksum, generated\_at)

ShareLink(timetable/version pointer, section optional, token, active, expires\_at, always\_latest)

#### HARD VALIDATION CONSTRAINTS

- No section double booking.
- No faculty double booking across all sections/departments.
- No room double booking.
- Faculty availability must be respected.
- Room/lab must be active and available.
- Room capacity must satisfy section strength unless an authorised override is stored.
- Room type/facilities must satisfy course/lab requirements.
- Lunch/common locked slots cannot be overwritten.
- Practical/lab block\_length must remain consecutive.

- A published version cannot be patched/deleted.
- Before publish, required weekly periods for every course offering must be complete or explicitly waived by an approver.

#### SOFT OPTIMISATION CONSTRAINTS

- Minimise faculty and section gaps.
- Avoid excessive consecutive lectures.
- Spread the same course across the week.
- Prefer same classroom for a section when possible.
- Respect faculty preferred slots.
- Balance room utilisation.
- Prefer appropriate-sized rooms.
- Support configurable weights for each preference.

#### AUTOMATIC GENERATION

Implement OR-Tools CP-SAT generation as a Celery task.

- Input: selected timetable version/scope, sections, course offerings, faculty availability, rooms, time slots and locked ScheduleEntry records.
- Locked entries become fixed constraints.
- Generate a valid solution satisfying hard constraints and optimising soft constraints.
- Persist solution only to DRAFT version.
- Return job status/progress plus solution score/warnings.
- Allow a configurable solver time limit.
- Include “regenerate only affected sections” or partial generation later; structure code so this is possible.

#### MANUAL BUILDER

Create /timetables/[id]/builder.

- Weekly grid with weekdays as rows and time slots as columns.
- Filter by session, semester, department, program, year, section and version.
- Click a cell to add/edit an entry; select course offering, class type, faculty, room and block length.
- Drag/move is optional initially but architecture should allow it.
- Show course short code + faculty initials + room in each cell.
- Conflict badges update after edits.
- Allow lock/unlock entry.
- Inspector panel shows periods required/scheduled/remaining, faculty availability and recommended free rooms.
- Save draft, validate, compare, submit for approval, publish, download PDF.
- Published version is read-only.

#### ROOM ALLOCATION

Build /room-allocation.

- Day/week filters.
- Room x period matrix or period x room matrix.
- Show occupied, free and blocked/maintenance states.
- Occupied cell shows section + course + faculty.
- Filter by building, floor, room type, capacity and department.
- “Find free room” endpoint/action based on day, slot/block, capacity and room type.
- Room chart must be calculated from ScheduleEntry; it must automatically change when the timetable changes.
- Export official Room Allotment Chart PDF.

#### VERSIONING AND PUBLICATION WORKFLOW

DRAFT -> VALIDATE -> PENDING\_APPROVAL (optional) -> APPROVED -> PUBLISHED.

When changing a published timetable, clone it into a new DRAFT version. Never edit PUBLISHED in place.

On publish:

1. Acquire a lock and run final validation.
2. Verify approval requirement.
3. Calculate structured diff against previous PUBLISHED version.
4. In one transaction mark new version PUBLISHED and previous PUBLISHED version SUPERSEDED.
5. After transaction commit, queue PDF generation and notification jobs.
6. Update any “latest published” share pointer.
7. Write audit log.

#### CHANGE DIFF

Return changes grouped by section/faculty/day.

Represent ADDED, REMOVED, MOVED and MODIFIED entries.

A diff item should include old/new day, slot, room, course and faculty where relevant.

Create a clear UI for “Previous vs New”.

#### NOTIFICATIONS

On every official publication:

- ALL active faculty receive a “New timetable version published” notification with effective date and download/latest link.
- Faculty whose timetable changed receive a detailed personalised diff.
- Class coordinators receive detailed changes for their sections and section PDF/share link.
- Approvers receive publication status summary.

**Channels:**

1. In-app notification centre.
  2. Email (MVP).
  3. Optional WhatsApp Cloud API adapter.
  4. Optional web push/PWA adapter.
- Use Celery retries and store delivery status.  
Add faculty "Acknowledge update" action.

**PDF/EXPORT**

Use server-side HTML/CSS templates + WeasyPrint.

**Exports:**

- Section timetable PDF matching the reference structure.
- Faculty personal timetable PDF.
- Room allocation PDF.
- Combined department/batch timetable PDF.

Every official PDF must include timetable version, effective date and generated timestamp.

Allow institution logo and authorised signature images, but keep public exports free of unnecessary private contact information.

**DISPLAY CODE**

Do not store strings like L/CAIT/SY/411 as the schedule source. Build display strings from structured data, e.g. {entry\_type\_short}/{course.short\_code}/{faculty.initials}/{room.code}

Make display-format rules configurable enough for lectures/practicals/common activities.

**API ENDPOINTS**

Implement REST endpoints for:

/auth/login, refresh, me  
/academic/sessions, semesters, programs, sections  
/courses, course-offerings  
/faculty and faculty/{id}/availability  
/rooms and /rooms/available  
/time-slot-templates and time-slots  
/timetables  
/timetables/{id}/versions  
/versions/{id}/entries  
/versions/{id}/validate  
/versions/{id}/generate  
/generation-jobs/{id}  
/versions/{id}/diff  
/versions/{id}/submit  
/versions/{id}/approve or reject  
/versions/{id}/publish  
/versions/{id}/exports/...  
/me/timetable  
/notifications and /notifications/{id}/acknowledge  
/audit

Use serializers with validation but move cross-model scheduling rules to service classes.

**NEXT.JS PAGES**

/login  
/dashboard  
/master/academic  
/master/courses  
/master/faculty  
/master/rooms  
/master/time-slots  
/timetables  
/timetables/[id]/builder  
/timetables/[id]/generate  
/timetables/[id]/validate  
/timetables/[id]/versions  
/room-allocation  
/faculty/me/timetable  
/notifications  
/reports  
/share/[token]

**BULK DATA SETUP**

Add CSV/XLSX import/export for faculty, courses/course offerings, sections and rooms. Return row-level validation errors. Imports must be transactional or support dry-run preview before commit.

**DASHBOARD**

Show:

- Current published version/effective date.

- Draft versions.
- Blocking conflicts.
- Pending approvals.
- Faculty update acknowledgements.
- Room utilisation.
- Latest timetable changes.

**TESTING**

Backend: pytest or Django TestCase. Include tests for faculty clash, room clash, section clash, capacity/type checks, locked entries, version immutability, publish transaction, diff calculation, notification recipients and permissions.

Frontend: unit/component tests for builder state, conflict display and permission-gated actions; add Playwright E2E for create draft -> edit -> validate -> approve -> publish -> faculty notification/download.

**SEED/DEMO DATA**

Provide a management command to seed one institution, one CSE department, multiple sections, 8 daily slots with a lunch break, faculty, courses and rooms/labs. Include enough sample data to demonstrate conflicts, room chart and timetable generation without copying personal phone numbers from the reference PDFs.

**IMPLEMENTATION ORDER**

Milestone 1: project setup, auth/RBAC, master data and imports.

Milestone 2: timetable/version/schedule models, validation engine and read-only grids.

Milestone 3: editable builder, room allocation and free-room finder.

Milestone 4: approval, publish, diff, audit and official PDF exports.

Milestone 5: in-app/email notifications and faculty acknowledgement.

Milestone 6: OR-Tools automatic generation and optimisation.

Milestone 7: reports, WhatsApp/push adapters and UX polish.

**CODING QUALITY**

- Use type hints and service classes in Django.
- Use DRF viewsets/actions only where appropriate; keep endpoints explicit for domain actions.
- Use select\_related/prefetch\_related and indexes.
- Use atomic transactions for multi-step writes.
- Use idempotent Celery tasks for PDFs/notifications.
- Use environment variables for secrets.
- Provide Docker Compose for local PostgreSQL + Redis.
- Add .env.example, README setup steps, migrations and seed command.
- Keep UI accessible and responsive.
- Do not mock core scheduling logic. Implement real conflict validation and a real room availability query.

**DELIVERABLE EXPECTATION**

First inspect the existing repository before changing code. Then present the intended file/module changes and implement milestone by milestone. Do not rewrite unrelated parts. After each milestone run backend tests, frontend typecheck/lint/build and explain any remaining issue. Keep migrations clean and include API examples for the major scheduling workflows.

## 25. Recommended First Sprint

| # | Task                                                                                                          |
|---|---------------------------------------------------------------------------------------------------------------|
| 1 | Create Django project/apps, PostgreSQL settings and RBAC base.                                                |
| 2 | Implement AcademicSession, Semester, Program, Section, Faculty, Course, CourseOffering, Room and TimeSlot.    |
| 3 | Create admin/API CRUD plus CSV import for faculty, rooms and course offerings.                                |
| 4 | Implement Timetable, TimetableVersion, ScheduleEntry and conflict validation services.                        |
| 5 | Build Next.js master-data screens and a read-only weekly timetable grid.                                      |
| 6 | Add timetable edit modal with course/faculty/room selection and real-time validation.                         |
| 7 | Add room allocation view derived from schedule entries.                                                       |
| 8 | Add clone-version, publish guard and audit log; automatic generator comes after these foundations are stable. |

## 26. Final Product Summary

The finished system should replace static timetable documents with a controlled scheduling workflow while still producing the familiar PDF format that faculty and students already understand. The single schedule database powers the section timetable, personal faculty timetable and room allocation chart. Conflict checking protects data quality; versioning protects

official history; notifications solve the communication problem after updates; and automatic generation reduces the coordinator's scheduling workload.

**Suggested project name:** Campus Scheduler, UniTimeFlow, AcademicGrid, ScheduleHub or TimeTableX. The codebase can keep a neutral internal name such as "academic\_scheduler".